

Universidad Nacional Autónoma de México

Facultad de Ciencias

Documentación de Pruebas IEEE 829

Proyecto Final: Sistema Básico de Administración de Configuración
(SBAC)

Materia: Pruebas de Software y Administración de la Configuración

Profesor: Luis Rey Rutiaga Robles

Integrantes:

Maryam Michelle Del Monte Ortega

Integrante 2

Integrante 3

Integrante 4

Fecha:

04 de junio de 2026

1. Plan de Pruebas

- **Identificador:** TP-SBAC-1.0
- **Introducción:** Este documento explica de forma clara cómo pusimos a prueba nuestro Sistema Básico de Administración de Configuración (SBAC). Nuestro objetivo es asegurar que esta herramienta de terminal (CLI) funcione sin problemas al guardar las versiones de los archivos.
- **¿Qué vamos a probar?:**
 - Que el repositorio se cree y se lea correctamente (`init`, `status`).
 - Que el sistema detecte y guarde los archivos sin perder información (`add`, `commit`).
 - Que el historial y las etiquetas sean fáciles de consultar (`history`, `baseline`, `list-baselines`, `diff`).
 - Que podamos viajar en el tiempo para recuperar archivos viejos (`checkout`).
- **¿Cómo lo vamos a probar?:** Usaremos un enfoque mixto. Revisaremos que el código interno haga lo que debe (caja blanca) y probaremos el programa como si fuéramos un usuario final (caja negra). Todo esto está automatizado con las herramientas `unittest` y `pytest`, y se ejecuta dentro de un contenedor Docker (`python:3.11-slim`) para que funcione igual en cualquier computadora.
- **Criterios de Éxito:** Consideramos que el sistema está listo si logra pasar automáticamente las 15 pruebas diseñadas y si la herramienta `pytest-cov` nos confirma que estamos evaluando más del 95 % de todo nuestro código escrito.

2. Especificaciones de Diseño de Pruebas

- **Identificador:** TDS-SBAC-1.0
- **Puntos clave a revisar:** Queremos estar seguros de que la carpeta `.sbac` no se corrompa, que el archivo de configuración `metadata.json` guarde bien los datos, y que el programa no colapse si el usuario escribe un comando equivocado.
- **Nuestra forma de trabajo:**
 - **División de tareas:** Separamos las pruebas en tres grupos: pruebas pequeñas para funciones individuales (unitarias), pruebas para ver cómo se comunican las funciones entre sí (integración), y pruebas para asegurar que los archivos físicos viajen bien en el tiempo (regresión).
 - **Entorno seguro:** Cada vez que corremos una prueba, creamos una carpeta temporal invisible (`tempfile.mkdtemp()`). Así podemos probar cómo se copian y borran los archivos sin poner en riesgo nuestro proyecto real.

3. Casos de Prueba

Aquí describimos los escenarios más importantes que evaluamos. *(Nota: Nuestro sistema automatizado corre un total de 15 pruebas para cubrir estos casos y varios más).*

- **TC-01: Iniciar el repositorio por primera vez**
 - **¿Qué necesitamos antes?:** Nada, solo una carpeta vacía.
 - **Pasos:** Escribir el comando `sbac start` o `sbac init`.
 - **Qué debería pasar:** El sistema debe crear una carpeta oculta `.sbac/`, un espacio para guardar los archivos (`snapshots/`) y un documento JSON listo para registrar los cambios.
- **TC-02: Evitar que el programa colapse por descuidos del usuario**
 - **¿Qué necesitamos antes?:** Estar en una carpeta donde NO se haya iniciado SBAC.
 - **Pasos:** Intentar usar comandos como `sbac add` o `sbac commit`.
 - **Qué debería pasar:** El programa debe detenerse amablemente y avisarle al usuario que primero debe iniciar el repositorio, en lugar de mostrar un error feo de Python en la pantalla.
- **TC-03: Comparar versiones usando etiquetas (Baselines)**
 - **¿Qué necesitamos antes?:** Tener un archivo modificado y guardado con dos etiquetas distintas (por ejemplo, “V1” y “V2”).
 - **Pasos:** Escribir `sbac diff V1 V2`.
 - **Qué debería pasar:** El sistema debe entender qué versión significa cada etiqueta y mostrarnos en pantalla exactamente qué líneas de código se agregaron o borraron.
- **TC-04: Viajar al pasado (Checkout)**
 - **¿Qué necesitamos antes?:** Tener un archivo guardado, haberlo modificado, y haber guardado esa nueva versión.
 - **Pasos:** Escribir `sbac checkout v1` para pedirle que regrese al estado original.
 - **Qué debería pasar:** El archivo actual debe sobrescribirse para volver a ser idéntico a la versión antigua que le pedimos.

4. Procedimientos de Prueba

- **Identificador:** TPR-SBAC-1.0
- **Requisitos Previos:** Solo necesitas tener Docker instalado en tu computadora.
- **Pasos para correr las pruebas:**

1. Abre tu terminal en la carpeta del proyecto SBAC.
2. Empaqueta el entorno de pruebas escribiendo:
`docker build -t sbac-tests .`
3. Inicia las pruebas automatizadas escribiendo:
`docker run -rm sbac-tests`
4. Observa la pantalla: verás la confirmación de que todas las pruebas pasaron en color verde y una tabla mostrando el porcentaje de código evaluado.

5. Informes de Incidentes

- **Identificador:** TIR-SBAC-01
 - **Fecha del problema:** 29 de mayo de 2026.
 - **¿Qué pasó?:** Durante las primeras pruebas, notamos que si intentábamos comparar dos versiones usando sus nombres de etiqueta (como `ESTABLE`), el programa colapsaba y se cerraba. Solo funcionaba si usábamos los números internos de versión (como `v2`).
 - **Problema causado:** No estábamos cumpliendo con una de las funciones principales que prometía el manual del usuario para visualizar cambios entre líneas base.
 - **Cómo lo arreglamos:** Modificamos la función `show_diff`. Ahora, antes de comparar, el sistema revisa si la palabra que escribió el usuario es una etiqueta, si lo es, la traduce silenciosamente a su número de versión interno y la comparación funciona perfecto. **Incidente cerrado.**
-
- **Identificador:** TIR-SBAC-02
 - **Fecha del problema:** 30 de mayo de 2026.
 - **¿Qué pasó?:** Al realizar pruebas de estrés en la función de seguimiento, detectamos que si un usuario ejecutaba `sbac add archivo.txt` múltiples veces seguidas, el archivo se registraba repetidamente en el arreglo del `metadata.json`.
 - **Problema causado:** Duplicidad de datos. Esto provocaba que al hacer un `commit`, el sistema intentara copiar el mismo archivo físico varias veces, consumiendo recursos innecesarios y causando comportamientos erráticos al leer el historial.
 - **Cómo lo arreglamos:** Se implementó una validación condicional en la función `add_file` (`if filename not in metadata["tracked_files"]`). Ahora el sistema verifica si el archivo ya está en seguimiento y, de ser así, omite el registro y avisa al usuario amablemente. **Incidente cerrado.**
-
- **Identificador:** TIR-SBAC-03

- **Fecha del problema:** 1 de junio de 2026.
- **¿Qué pasó?:** Durante las pruebas de caminos alternos (edge cases), se ejecutó el comando `sbac commit "Mensaje"` en un repositorio recién inicializado que no tenía ningún archivo agregado.
- **Problema causado:** Generación de “commits fantasma”. El sistema creaba un nuevo ID de versión y generaba una carpeta vacía dentro de `snapshots/`, alterando el orden del historial sin guardar información real.
- **Cómo lo arreglamos:** Se agregó una barrera preventiva al inicio de la función `create_commit`. Si la lista de archivos en seguimiento está vacía, la ejecución se aborta inmediatamente y se notifica al usuario que debe usar `sbac add` primero. **Incidente cerrado.**

6. Informes de Prueba

- **Identificador:** TSR-SBAC-1.0
- **Resumen del trabajo:** Corrimos todo el paquete de pruebas sobre la versión final del sistema sin ningún contratiempo. Para tener el código más limpio y ordenado, separamos las pruebas en tres archivos distintos (`test_unit.py`, `test_integration.py` y `test_regression.py`).
- **Nuestros Resultados:**
 - **Pruebas realizadas:** 15
 - **Pruebas que pasaron exitosamente:** 15 (100 % de éxito).
 - **Porcentaje de código evaluado:** 99 % (150 de 152 líneas de código).
- **¿Por qué sacamos 99 % y no 100 %?:** Alcanzar un 99 % es un indicador de calidad excelente. El 1 % faltante son solo dos líneas de código interno (132 y 134) que actúan como “seguros de vida” al momento de traducir las etiquetas de las versiones. Es muy difícil obligar al entorno de pruebas a activar estas alarmas preventivas, lo cual es algo completamente normal y aceptado en la industria del software.
- **Conclusión Final:** El sistema SBAC cumple con su propósito de guardar versiones de forma confiable. Demostró ser muy resistente incluso cuando intentamos engañarlo borrando archivos a la fuerza o pidiéndole datos vacíos. Al usar Docker, garantizamos que el profesor o cualquier usuario pueda usarlo sin tener que configurar nada en su computadora. El software está listo para entregarse.